



Licenciatura em Engenharia Informática e Multimédia

Processamento de Imagem e Visão

2º Trabalho Laboratorial – Contagem de Veículos em Imagens de Vídeo

Trabalho Realizado por:

Martim Ramos - A51736

Guilherme Graça - A51827

Turma 51D

Docente: Pedro Mendes Jorge

7 de Dezembro 2025

ENUNCIADO A

Índice

1. Introdução
2. Estimação de Imagem de Fundo
3. Detecção de Pixeis Ativos
4. Operações Morfológicas
5. Detecção de Regiões Ativas
6. Correspondência de regiões entre Imagens Consecutivas
7. Contextualização do Objetivo Opcional
8. Calibração e Parâmetros Reais
9. Transformação de Perspetiva (Homografia)
10. Configuração das Vias e FPS
11. Algoritmo de Cálculo de Velocidades
12. Conclusão
13. Bibliografia

Introdução

Este trabalho tem como objetivo desenvolver um sistema de visão por computador que seja capaz de detectar e contar veículos em um vídeo de uma autoestrada.

A abordagem que implementamos segue um pipeline básico de processamento de vídeo, ele segue os seguintes passos:

- **Estimação do Fundo** : Criamos uma imagem de estática referência (neste caso a estrada) para depois isolarmos os veículos.
- **Deteção de Pixels Ativos** : Utilizamos técnicas de diferença de imagem e de threshold para segmentar os veículos do fundo.
- **Utilização de operadores morfológicos** : Aplicação de operadores para reduzirmos o ruído e consolidar as formas dos componentes conexos.
- **Definição de Região de Interesse** : Restringimos a área que estamos a analisar de modo a reduzir os falsos positivos.
- **Extração de Componentes Conexos** : Identificamos os veículos/regiões ativas e calculamos as suas propriedades geométricas como as bounding boxes e centroides.
- **Tracking** : Associamos temporalmente os frames consecutivos de modo a termos uma contagem consistente

IMPORTS

```
In [14]: import numpy as np
import cv2
import matplotlib.pyplot as plt
import math
```

FUNÇÕES UTILITÁRIAS

De modo a modularizar o código, criamos duas funções auxiliares que nos ajudam a organizar o código em toda a sua extensão. A primeira função, 'reproduzir_frames', é responsável por reproduzir os vídeos teste com todas as nuances que estes precisam (região de interesse, contornos dos veículos e a indexação de cada um deles). A segunda função, 'video_para_frames', centraliza o processo de conversão do vídeo em frames, ou seja, recebe um caminho de um vídeo e converte este para uma lista de frames. Esta função é útil para depois usar na função anteriormente mencionada a lista de frames convertida.

Estas funções foram criadas de modo a não ter-se que repetir o mesmo código constantemente nas várias células que precisam de fazer uso delas.

```
In [15]: #Função para fazer os outputs
def reproduzir_frames(lista_frames_base, pontos_roi=None, dados_rastreamento=None):
    print(f"A reproduzir '{titulo}'")
    print("Pressiona a tecla 'Q' para fechar")
```

```

for i, frame in enumerate(lista_frames_base):
    frame_visualizacao = frame.copy() #Fazemos copia para não esmagar o video

    #Desenha a Regiao de Interesse (Se houver)
    if pontos_roi is not None:
        cv2.polylines(frame_visualizacao, [pontos_roi], isClosed=True, color

    #Desenha Caixas e IDs dos Carros (Se houver dados para este frame)
    if dados_rastreamento is not None and i < len(dados_rastreamento):
        veiculos_frame_atual = dados_rastreamento[i]

        for dados_veiculo in veiculos_frame_atual:
            #Se tiver 4 valores é só detecção se tiver 5 valores então tem o
            #primeiros 4 para desenhar a caixa
            x, y, w, h = dados_veiculo[:4]

            cv2.rectangle(frame_visualizacao, (x, y), (x + w, y + h), (0, 0,

            # Se tivermos o 5o elemento (o ID), desenha se o texto
            if len(dados_veiculo) == 5:

                cv2.putText(frame_visualizacao, f"V{dados_veiculo[4]}", (x,
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 0), 2)

        #Mostra o resultado
        cv2.imshow(titulo, frame_visualizacao)

        if cv2.waitKey(45) & 0xFF == ord('q'): #30ms entre frames
            break

cv2.destroyAllWindows()
print("Reprodução terminada.")

#Pega no video e transforma num array de frames - util para usar a função de cim
def video_para_frames(caminho_video):
    cap = cv2.VideoCapture(caminho_video)
    frames = []

    if not cap.isOpened():
        print("ERRO - Não foi possível abrir o vídeo!")
        return []

    while True:
        bReturn, frame = cap.read()
        if not bReturn:
            break
        frames.append(frame)

    cap.release()
    print(f"Vídeo carregado - {len(frames)} frames")
    return frames

```

Estimação de imagem de fundo

A estimação de uma imagem de fundo limpa é muito importante para sistemas como este de videovigilância com câmara fixa. Para detetar objetos em movimento em um situação em que o fundo é constante a estratégia mais usada é a de Subtração do

Fundo. Mas para fazermos isso é necessário termos um modelo de referencia do cenário vazio, que neste caso é só a estrada.

Tendo em conta que não nos é fornecido uma imagem da estrada sem nenhum carro, então temos que estimar a imagem temporalmente. Para isso usamos uma filtragem temporal com filtro de mediana.

Se calcularmos a mediana de cada fixel ao longo de todos os frames, então conseguimos remover os objetos que passam rapidamente. Isto porque em média cada pixel passa mais tempo a mostrar "o fundo" do que a mostrar o carros.

```
In [16]: video_original_frames = video_para_frames("AutoEstrada.avi")#converte o video pa

def estimar_fundo(lista_frames):
    if not lista_frames:
        print("Erro: A lista de frames está vazia.")
        return None

    print(f"A calcular a mediana de {len(lista_frames)} frames...")

    # Calcula a mediana ao longo do tempo (no axis=0)
    # A lista de frames é convertida logo para um array numpy aqui
    fundo = np.median(lista_frames, axis=0).astype(dtype=np.uint8)

    return fundo

imagem_fundo = estimar_fundo(video_original_frames)

# 3. VISUALIZAÇÃO
if imagem_fundo is not None:
    plt.figure(figsize=(10, 6))
    plt.imshow(cv2.cvtColor(imagem_fundo, cv2.COLOR_BGR2RGB))
    plt.title("Imagem de Fundo (Calculada em Memória)")
    plt.axis('off')
    plt.show()
```

Vídeo carregado - 234 frames

A calcular a mediana de 234 frames...

Imagem de Fundo (Calculada em Memória)



Na estimação de imagem de fundo, decidimos percorrer os frames todos sendo que o vídeo em análise não é muito grande, mas para vídeos de maior dimensão teríamos de optar por outra estratégia de modo a não comprometer o custo computacional da máquina que correrá o código. Opções possíveis seriam: Percorrer os frames do vídeo de passo em passo (ex: 2 em 2, 5 em 5, etc) ou selecionar aleatoriamente um conjunto de frames do vídeo.

Resultado Obtido : A imagem que resulta apresenta só a autoestrada sem qualquer veículo. Este é o resultado que estávamos à espera porque em qualquer posição (x,y) da imagem, a presença do carro é temporalmente curto quando comparamos com o tempo em que se vê a autoestrada. Podemos concluir que o filtro de mediana elimina bem os elementos atípicos temporalmente.

Deteção de Pixeis Ativos

- Algoritmo com 5 passos:
 1. Calcular imagem binária (pixels activos)

$$I_{out}(r, c) = \begin{cases} 1 & \text{se } |I_n(r, c) - I_{n-k}(r, c)| > \tau \\ 0 & \text{caso contrário} \end{cases}$$

Para realizar a detecção de pixels ativos, decidimos basear-nos na diferença absoluta entre o frame atual e a imagem de fundo presente nos slides (Capítulo 7) lecionados pelo professor. Primeiramente convertemos a imagem de fundo anteriormente estimada para a escala de cinzentos bem como todos os frames do vídeo. De seguida, convertemos tanto a imagem de fundo como os frames para float de modo a permitir números negativos corretamente na subtração. Posteriormente, aplicamos a fórmula da diferença absoluta. Por fim, aplicamos o limiar para determinar quais os pixels que são considerados ativos (valores acima do limiar) e quais os que não são (valores abaixo do limiar).

Quanto ao valor do limiar que decidimos utilizar, é de boa prática não usar um valor extraído por tentativa e erro, mas sim um valor calculado com base nos dados do vídeo. Para isso, baseamo-nos na estatística da imagem de fundo, mais especificamente no desvio padrão dos valores dos pixels da imagem de fundo. Assim teremos uma noção da complexidade ou contraste natural da autoestrada presente no vídeo. Usaremos 50% do desvio padrão como limiar pois o desvio padrão total inclui o alto contraste das linhas brancas presentes no vídeo, o ruído que queremos filtrar é tipicamente inferior a metade dessa variação global. Portanto, o limiar a utilizar é adaptativo em relação ao conteúdo do vídeo, o que é mais robusto do que um valor fixo arbitrário.

Este limiar é definido dinamicamente dentro da função permitindo assim que o limiar se adapte e seja diferente para diferentes vídeos.

```
In [17]: def detetar_pixels_ativos(lista_frames, imagem_fundo):

    if not lista_frames:
        print("Erro: A lista de frames está vazia.")
        return []

    #Garantimos que o fundo está em grayscale
    if len(imagem_fundo.shape) == 3:
        fundo_gray = cv2.cvtColor(imagem_fundo, cv2.COLOR_BGR2GRAY)
    else:
        fundo_gray = imagem_fundo

    #Convertemos para float - para depois usar o np.abs() - se deixarmos em uint
    fundo_float = fundo_gray.astype(float)

    ##CALC DO LIMIAR
    sigma = np.std(fundo_float) #Desvio padrão global
    limiar = sigma * 0.5 #Usamos 50% do desvio padrão por causa do alto contrast

    print(f"Debug:")
    print(f"Limiar Calculado: {limiar:.2f}")
    print(f"-----")

    lista_frames_processados = []

    #percorre os frames um a um
    for frame in lista_frames:
        #Passa frame para grayscale e float
        frame_cinzento = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        frame_float = frame_cinzento.astype(float)
```

```

#Faz a diferença absoluta |Fundo - Frame|
diferenca_abs = np.abs(fundo_float - frame_float)

# Aplica Limiar
pixeis_ativos = np.zeros_like(diferenca_abs, dtype=np.uint8)#array de zeros
pixeis_ativos[diferenca_abs > limiar] = 255 #Se a diferença for maior q

lista_frames_processados.append(pixeis_ativos)#guarda

#Retorna os frames com 0 os pixels inativos e 1 os pixels ativos
return lista_frames_processados

#_____OUTPUT-TESTE_____
frames_pixeis_ativos = detetar_pixeis_ativos(video_original_frames, imagem_fundo)
print("PIXEIS ATIVOS")
reproduzir_frames(frames_pixeis_ativos, titulo="Resultado - Pixeis Ativos")

```

Debug:

Limiar Calculado: 23.23

PIXEIS ATIVOS

A reproduzir 'Resultado - Pixeis Ativos'

Pressiona a tecla 'Q' para fechar

Reprodução terminada.

Resultado Obtido : O output desta fase é uma sequencia de imagens binárias onde os veículos aparecem como aglomerados de pixels brancos e o fundo a preto. No entanto temos algum problemas, nomeadamente a presença de vários pontinhos brancos dispersos. Podemos assumir que este ruído deve acontecer devido a pequenas variações de iluminação, vibrações da câmara ou movimentos de elementos que são "estáticos" como as árvores ao fundo.

Estes artefactos tem que ser filtrados posteriormente para evitarmos que também sejam classificados como veículos.

Operadores Morfológicos

Após a deteção de pixels ativos, podemos observar algum ruído no conjunto de imagens binárias obtidas (vídeo) como por exemplo, pequenos pontos que correspondem às linhas a trajetado das vias onde os carros circulam e de algumas árvores presentes no vídeo que ao apresentarem pouco mas algum movimento no vídeo são detetadas também como pixels ativos. Para mitigar este ruído de forma a que no futuro não seja classificado , e apenas sim os veículos, precisamos de aplicar alguns operadores morfológicos.

```

In [18]: def aplicar_operadores_morfologicos(video_entrada):
#Recebe uma lista de frames binários correspondente aos pixels ativos.

frames_operadores_morfologicos = [] #lista onde se vai guardar os frames dos

#ELEMENTOS ESTRUTURANTES/KERNELS
#Circulo pequeno para eliminar os pontinhos de ruído em interferir muito com
kernel_ruído = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))

```

```

#Retangulo um pouco maior para deixar os carros mais cheios assim não correm
#RECT porque os carros são retangulares
kernel_uniao = cv2.getStructuringElement(cv2.MORPH_RECT, (8, 8))

for frame in video_entrada:

    # OPEN - Remove os pontinhos brancos isolados de ruído
    frame_sem_ruído = cv2.morphologyEx(frame, cv2.MORPH_OPEN, kernel_ruído)

    #CLOSE - Preenche buracos dentro dos objetos brancos (carros)
    frame_solido = cv2.morphologyEx(frame_sem_ruído, cv2.MORPH_CLOSE, kernel_uniao)

    frames_operadores_morfologicos.append(frame_solido)

return frames_operadores_morfologicos

# _____ OUTPUT-TESTE _____
video_morfologia = aplicar_operadores_morfologicos(frames_pixeis_ativos)
print("OPERADORES MORFOLÓGICOS APLICADOS")
reproduzir_frames(video_morfologia, titulo="Resultado - Operadores")

```

OPERADORES MORFOLÓGICOS APLICADOS

A reproduzir 'Resultado - Operadores'

Pressiona a tecla 'Q' para fechar

Reprodução terminada.

A estratégia que nós adotamos envolveu o uso de dois passos de aplicação de operadores morfológicos diferentes com elementos estruturantes diferentes:

OPEN :

- A operação consiste em uma erosão seguida de uma dilatação
- Utilizamos como elemento estruturante um círculo pequeno (Elipse 3x3) - Mais adequado para os pontinhos pequenos
- Como objetivo desta aplicação queremos remover os pixels isolados a branco (pontinhos) de ruído que foram gerados no passo anterior com a erosão e a dilatação vai servir para recuperar a forma como os carros foram afetados por essa erosão. Abaixo temos uma imagem que exemplifica a utilização deste operador.



CLOSE :

- A operação consiste em uma dilatação seguida de uma Erosão

- Utilizamos como elemento estruturante um quadrado maior 8x8 (Rect 8x8) - Adequado tendo em conta o formato rectangular dos veiculos.
- Como objetivo desta aplicação queremos tentar preencher os buracos que estão dentro das regiões dos veiculos e também tentar juntar os carros que se fragmentaram para depois ser mais facil de fazer a sua detecção. Abaixo temos uma imagem que exemplifica a utilização deste operador.



Resultado Obtido : Após aplicarmos os operadores necessários, podemos ver que no vídeo resultante, o ruído foi reduzido significativamente, mantendo apenas assim os pixels ativos que queremos classificar (veículos). Podemos visualizar regiões brancas e compactas que correspondem aos veiculos. Um efeito secundário que obtivemos e que é bastante notavel é que devido à perspectiva, os veiculos muito distantes e proximos uns dos outros acabam por se fundir em uma unica região conexa na operação de CLOSE. No entanto tendo em conta o proximo passo da definição da Região de Interesse em que os carros estão mais proximos da câmara e separados, esse efeito acaba por ser irrelevante.

Deteção de Regiões Ativas

A nossa ideia foi criar uma região de interesse (ROI) que abranja todas as vias da autoestrada, de modo a que apenas os veículos que passem por essa região sejam posteriormente contabilizados. Definimos os valores de cada um dos 4 pontos do trapézio "à mão" com base nas proporções da estrada no vídeo. Tentamos nos aproximar o maximo possivel dos exemplos visuais presentes no enunciado. Este passo é importante para evitar processar veiculos que passem na faixa de rodagem oposta, ruido que tenha sobrado e veiculos distantes que estão menos segmentados.

```
In [19]: #Defenição dos 4 cantos do polígono onde vamos detetar os carros
#(0,0) - canto superior esquerdo

#Canto Inferior Esquerdo
p1 = [25, 220]

#Canto Superior Esquerdo
p2 = [98, 80]

#Canto Superior Direito
p3 = [160, 80]

#Canto Inferior Direito
p4 = [215, 220]
```

```
#array com os pontos da nossa região de interesse
pontos_roi = np.array([p1, p2, p3, p4], np.int32)

#_____OUTPUT-TESTE_____
reproduzir_frames(video_original_frames, pontos_roi, titulo="Teste da Regiao de
```

A reproduzir 'Teste da Regiao de Interesse'

Pressiona a tecla 'Q' para fechar

Reprodução terminada.

Podemos observar um output com o vídeo original com a região de interesse desenhada a verde.

Com a região de interesse definida corretamente, podemos agora avançar para a fase de detectar as regiões ativas (veículos) que passam por essa região criada anteriormente.

```
In [20]: def extrair_componentes_conexos(video_binario, pontos_roi):

    lista_bounding_boxes = []

    for frame_bin in video_binario:
        #lista das bounding boxes / caixas que envolvem os carros em 1 frame
        caixas_neste_frame = []

        #EXTRAIMOS OS COMPONENTES CONEXOS
        #RETR_EXTERNAL - Ignora buracos dentro dos carros que temos ainda depois
        #CHAIN_APPROX_SIMPLE - guarda só os pontos essenciais
        contornos, _ = cv2.findContours(frame_bin, cv2.RETR_EXTERNAL, cv2.CHAIN_

        for contorno in contornos:
            x, y, w, h = cv2.boundingRect(contorno)#pega no retangulo q envolve

            #Calcula o centro do retangulo
            centro_x = int(x + w / 2)
            centro_y = int(y + h / 2)

            #pointPolygonTest devolve se um ponto está dentro ou fora de um poli
            #valor positivo - está dentro ; valor negativo - está fora ; 0 - est
            #o parametro false diz à função que não precisa de calcular a distan
            if cv2.pointPolygonTest(pontos_roi, (centro_x, centro_y), False) >=
                #Se estiver dentro então guardamos
                caixas_neste_frame.append((x, y, w, h))

            #Guardamos a lista de carros de cada frame na lista de frames
            lista_bounding_boxes.append(caixas_neste_frame)

    return lista_bounding_boxes

#_____OUTPUT-TESTE_____
dados_veiculos = extrair_componentes_conexos(video_morfologia, pontos_roi)

print("Extração dos Componentes Conexos")
#VISUALIZAR COM O VIDEO ORIGINAL
reproduzir_frames(video_original_frames, pontos_roi, dados_veiculos, titulo = "R

#VISUALIZAR COM O VIDEO DOS OPERADORES MORFOLOGICOS
#video_morfologia_rgb = [cv2.cvtColor(frame, cv2.COLOR_GRAY2BGR) for frame in vi
#reproduzir_frames(video_morfologia_rgb, pontos_roi, dados_veiculos, titulo = "R
```

Extração dos Componentes Conexos

A reproduzir 'Resultado da Extracao dos Componentes'

Pressiona a tecla 'Q' para fechar

Reprodução terminada.

Nesta fase fizemos a etiquetagem de componentes conectos em que o objetivo é agrupar os pixels ativos vizinhos em objetos distintos.

Para extrairmos os componentes conexos dos pixels ativos, decidimos utilizar a função **cv2.findContours** da biblioteca OpenCV. Esta função é eficiente e amplamente utilizada para detetar contornos de componentes conexos em imagens binárias.

Como parametro do findContours utilizamos:

- **cv2.RETR_EXTERNAL**: Este parametro é importante na nossa implementação pois ele recupera os contornos externos e ignora os buracos dentro dos veiculos, buracos estes que são possíveis observar em alguns momentos depois da aplicações dos operadores.
- **cv2.CHAIN_APPROX_SIMPLE**: Este é um parametro que é obrigatório preencher do findContours, esta opção faz com que apenas seja guardado pontos essenciais, desta forma otimiza o uso de memória.

Ao aplicar esta função às imagens processadas (após a aplicação dos operadores morfológicos), conseguimos identificar as diferentes regiões ativas que correspondem aos veículos presentes na região de interesse.

Para cada contorno que detetamos, calculamos a sua bounding box / caixa envolvente e o seu respectivo centroide(ponto central da caixa).

Usamos os centroides das bounding boxes calculadas para representar a posição dos veículos em cada frame de modo a poder observar os contornos ao longo de toda a extensão da região de interesse. Recorremos à função **cv2.pointPolygonTest** para verificar se o centroide de cada caixa/boundingbox se encontra dentro da região de interesse. Se o centroide estiver dentro da região ou na sua borda. Se esta condição verificar-se desenhmos o contorno no frame atual. Esta verificação é feita para cada frame do vídeo. Se não usassemos esta verificação, outros veiculos da faixa de rodagem oposta e tambem as junções de veiculos (quando está muito longe da camara) verificadas no passo anterior seriam detetadas ou até mesmo pontos de ruído que não tenham sido totalmente eliminados pelos operadores morfológicos aplicados anteriormente. Quando os veículos se ausentam da região de interesse, os contornos deixam de ser desenhados. Todos os veículos que circulem fora da região de interesse são totalmente ignorados para efeitos de contagem.

Resultado Obtido : Podemos ver que as Bounding Boxes desenhadas acompanham corretamente os veiculos apenas dentro dos limites da região de interesse. Para efeitos de teste e nitidez do ambiente envolvente, usamos o video original como base para desenhar os contornos dos veículos detetados, como fizemos anteriormente com a região de interesse.

No entanto verificamos que por breves momentos quando os veiculos estão a sair da nossa região de interesse pode acontecer uma intermitência em que visualmente as caixas "piscam". Isto acontece porque o centroide oscila momentaneamente para dentro e para fora da ROI, seja por causa do redimensionamento da caixa ou do seu posicionamento, isto faz com que a caixa desapareça e reapareça em frames seguidos. Logo este é um problema que temos que resolver no passo seguinte para termos um bom rastreamento dos veiculos.

De seguida passamos à fase da contagem dos veículos.

Correspondência de regiões entre Imagens Consecutivas

Após termos realizado a deteção de regiões ativas, o próximo passo é estabelecer a correspondência entre essas regiões em imagens consecutivas. Para isso, decidimos que usar um método baseado na distância entre as boundingboxes das regiões ativas em frames consecutivos seria uma boa ideia para conseguir distinguir cada veículo. Isto porque a contagem de veiculos temos que saber que o objeto A no frame t é o mesmo objeto A no frame $t+1$.

```
In [21]: #Constantes de controlo da função

MAX_DIST_ENTRE_BBOXES = 25
FRAMES_PRESISTENCIA = 30

def correspondencia_imgs_consecutivas(lista_bboxes_por_frame):
    resultado_rastreamento = []
    proximo_id_disponivel = 1

    carros_ativos = {} #Veiculos detectados no frame anterior
    carros_perdidos = {} #Veiculos que desapareceram do frame á menos de "FRAMES_

    for idx, caixas_no_frame in enumerate(lista_bboxes_por_frame):

        #PREPARAÇÃO
        veiculos_atuais = [] #veiculos detectados no frame que se está a percorrer
        for (x, y, w, h) in caixas_no_frame:
            veiculos_atuais.append({
                'box': (x, y, w, h),
                'position': (x, y), #Assumimos o canto superior esquerdo como a
                'id': None
            })

        #FRAME 0
        #Inicialmente detectamos todos os veiculos presentes no frame inicial
        #e ordenamos de modo a que os que estão mais longe da camara têm um indi
        if idx == 0:
            lista_para_ordenar = [] #array temporario para ordenar os carros no

            for veiculo in veiculos_atuais:
                # Usamos o Y da posição
                y_topo = veiculo['position'][1]
                lista_para_ordenar.append((y_topo, veiculo))
```

```

lista_para_ordenar.sort()#ordena pela altura

# Redefinimos o array de veiculos_atuais já ordenado
veiculos_atuais = []

for y, veiculo in lista_para_ordenar:
    veiculos_atuais.append(veiculo)

# Atribuimos os primeiros IDs
for veiculo in veiculos_atuais:
    veiculo['id'] = proximo_id_disponivel
    proximo_id_disponivel += 1

# FRAMES SEGUINTEs - Distância e Persistência
else:
    # Primeiro identificamos os carros tendo em conta a sua distancia re
    ids_ja_associados = []
    detecoos_ja_associadas = []

    #percorremos os carros anteriores
    for id_antigo, dados_antigos in carros_ativos.items():

        melhor_dist = MAX_DIST_ENTRE_BBOXES
        melhor_indice = -1 #inicialmente nenhum

        pos_antiga = np.array(dados_antigos['position'])#convetemos para
        #comparamos a distancia dos carros do frame anterior aos do fram
        for i, veiculo in enumerate(veiculos_atuais):

            if i in detecoos_ja_associadas:
                continue

            pos_atual = np.array(veiculo['position'])
            dist = math.dist(pos_atual, pos_antiga)
            if dist < melhor_dist:
                melhor_dist = dist #se for menor então atualizamos
                melhor_indice = i

            if melhor_indice != -1:
                veiculos_atuais[melhor_indice]['id'] = id_antigo
                ids_ja_associados.append(id_antigo)
                detecoos_ja_associadas.append(melhor_indice)

    # Depois tentamos corresponder se um carro que "apareceu" está muito
    #Se sim, significa que é o mesmo carro e brevemente ele deixou de se
    for i, veiculo in enumerate(veiculos_atuais):

        if veiculo['id'] is None:
            melhor_dist_perdido = MAX_DIST_ENTRE_BBOXES
            id_recuperado = None
            pos_atual = np.array(veiculo['position'])

            for id_perdido, dados_perdidos in carros_perdidos.items():

                pos_perdida = np.array(dados_perdidos['position'])
                dist = math.dist(pos_atual, pos_perdida)
                if dist < melhor_dist_perdido:
                    melhor_dist_perdido = dist

```

```

        id_recuperado = id_perdido

        if id_recuperado is not None:

            veiculo['id'] = id_recuperado

            # Como não podemos apagar itens do dicionário enquanto i
            # fazemos o del logo a seguir (aqui é seguro pois temos
            del carros_perdidos[id_recuperado]

        #Cria se IDs para quem é realmente novo
        for veiculo in veiculos_atuais:
            if veiculo['id'] is None:
                veiculo['id'] = proximo_id_disponivel
                proximo_id_disponivel += 1

        # ATUALIZAÇÃO DE ESTADO
        novos_carros_ativos = {}
        output_frame = []

        # Guardar o estado atual dos detetados
        for veiculo in veiculos_atuais:
            id_carro = veiculo['id']
            box = veiculo['box']
            posicao = veiculo['position']

            novos_carros_ativos[id_carro] = {'position': posicao, 'box': box}
            output_frame.append((box[0], box[1], box[2], box[3], id_carro))

        #Gere quem desaparece (Frames > 0)
        if idx > 0:

            for id_antigo, dados_antigos in carros_ativos.items():
                if id_antigo not in novos_carros_ativos:
                    # Move para a lista de perdidos
                    carros_perdidos[id_antigo] = {
                        'position': dados_antigos['position'],
                        'ttl': FRAMES_PRESISTENCIA
                    }

            #Decrementa o TTL e remove os que passaram
            ids_para_remover = []
            for id_perdido in carros_perdidos:
                carros_perdidos[id_perdido]['ttl'] -= 1
                if carros_perdidos[id_perdido]['ttl'] <= 0:
                    ids_para_remover.append(id_perdido)

            for id_rem in ids_para_remover:
                del carros_perdidos[id_rem]

            # Atualiza o dicionário principal
            carros_ativos = novos_carros_ativos
            resultado_rastreamento.append(output_frame)

        return resultado_rastreamento

#OUTPUT-TESTE_____

print("Correspondencia de Regiões entre imagens consecutivas")
dados_com_veiculos_identificados = correspondencia_imgs_consecutivas(dados_veicu
reproduzir_frames(video_original_frames, pontos_roi, dados_com_veiculos_identifi

```

Correspondencia de Regiões entre imagens consecutivas
A reproduzir 'Resultado Final'
Pressiona a tecla 'Q' para fechar
Reprodução terminada.

O nosso algoritmo implementado neste passo é baseado na distancia espacial e memoria temporal:

- Correspondencia de veiculos através da Distancia Euclidiana:

Para isto partimos do pressuposto que quando a taxa de frames é alta que chegue, o deslocamento de um veiculo entre frames é pequeno.

Calculamos a distancia Euclidean entre a posição de um veiculo (canto superior esquerdo da boundingbox - utilizamos esta posição apenas por conveniencia, não havia necessidade de voltar a calcular o centroide) no frame atual com a posição dos veiculos registrados no frame anterior. Usamos a distancia Euclidean porque esta é uma métrica comumente usada para medir a distância entre dois pontos num espaço bidimensional, calculamos através da função **math.dist** que já tinhamos prática na sua utilização.

Se a distancia calculada for inferior a **MAX_DIST_ENTRE_BBOXES** (constante esta que foi definida como 25 pixels pois esta é uma distancia curta que chegue para não assumir um carro que esteja na via do lado ou atrás/frente como o mesmo) então assumimos que é o mesmo veiculo e mantemos o seu ID.

- Gestão das falhas de persistência:

Para resolvermos o problema falado no passo anterior de intermitencia quando os veiculos saem da ROI, decidimos implementar um sistema de time to live.

Em suma, quando um veiculo com ID já atribuido deixa de ser detetado, não o eliminamos logo, em vez disso movemos o para uma lista de veiculos que acabaram de "desaparecer". Se um veiculo novo for detetado dentro de um espaço de 30 frames (**FRAMES_PRESISTENCIA** - tempo curto de time to live que chegue para não assumir o proximo carro a sair na mesma via como um que acabou de sair) depois de um veiculo "desaparecer" e estiver muito perto da ultima zona em que esse carro desaparecido saiu da ROI então sabemos que é o mesmo. Isto funciona porque os novos carro que são detetados estão muito mais perto da câmara do que os carros que acabaram de sair.

- Ordenação Inicial

Ao longo do desenvolvimento da função notamos que inicialmente, o carro que está à frente estava a ser identificado como V4 e os que estavam atrás como V1,V2 e V3. Para corrigir isto, tivemos a necessidade de no primeiro frame, ordenar os carros de forma correta pois o resultado que queremos, e que faz mais sentido, é que o carro mais à frente na zona da região de interesse tenha sido o primeiro a entrar na mesma portanto queremos que este seja o veículo 1 (V1) e depois assim sucessivamente para os restantes carros presentes no frame 0. Esta ordenação é feita com base na coordenada y de cada veículo, ordenando do menor para o maior valor de y (de cima para baixo na imagem).

Resultado Final : O algoritmo que desenvolvemos podemos observar que consegue seguir os veículos ao longo da nossa Região de Interesse e mantém os IDs de cada veículo estáveis o que permite uma boa contagem dos veículos no fluxo de tráfego.

OBJETIVO OPCIONAL - Estimar a velocidade dos veículos e a velocidade média em cada via da autoestrada

Contextualização do Objetivo Opcional

Um dos desafios em sistemas de visão aplicados ao tráfego é a extração de métricas como velocidade e distância a partir de imagens 2D.

O problema é a Projeção de Perspetiva, onde a escala da imagem varia com a profundidade, ou seja, um objeto distante ocupa menos pixels do que um objeto próximo, mesmo que percorra a mesma distância. Para resolver este problema e calcular a velocidade real em km/h dos veículos, e também a velocidade média por via, implementámos uma transformação geométrica (Homografia) de modo a obter uma vista de topo.

Com base em informações do IMT sabemos que a largura de cada via numa autoestrada é de 3,75 metros, como podemos ver na imagem abaixo:

Quadro 2 – Características técnicas das auto-estradas interurbanas

Nível de serviço	Velocidade máxima de circulação	Velocidade base	Parâmetros Geométricos							Distância entre nós
			Planta	Perfil Longitudinal			Perfil Transversal			
			Raio mínimo	Inclinação máxima	Raio convexo mínimo	Raio côncavo mínimo	Largura das vias	Separador Central	Berma	
B	120 km/h	140 km/h	1 200 m	3%	20 000 m	8 000 m	3,75 m	≥ 4,00 m	Esq 1,50 m Dir 3,00 m	8 km
C		120 km/h	700 m	4%	17 000 m	7 500 m		≥ 3,00 m		

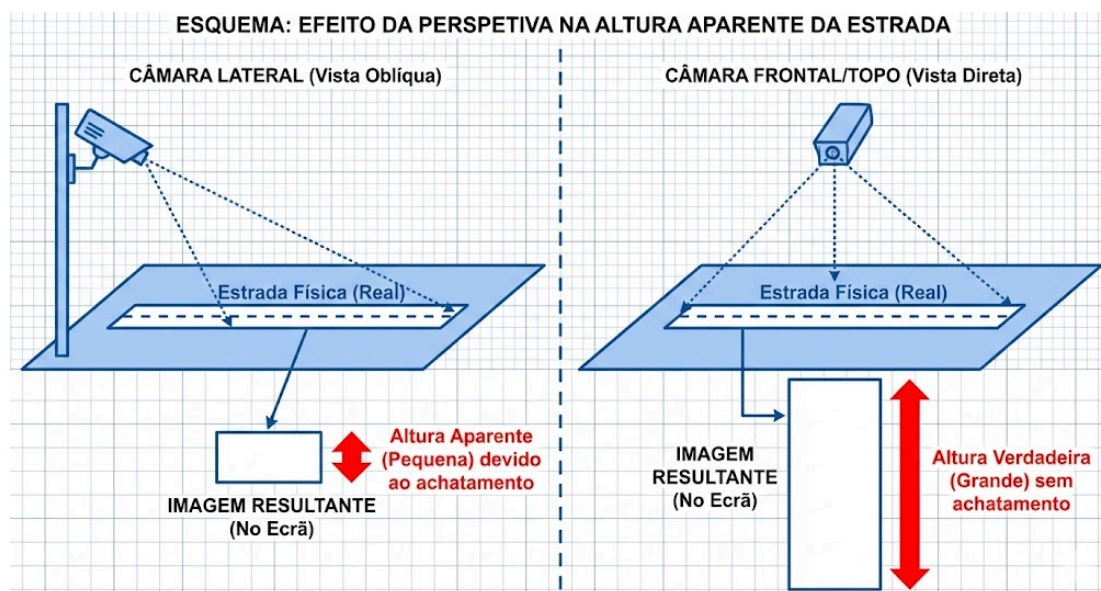
Com base em informações retiradas do Manual de Sinalização e Planeamento das Acessibilidades e da Gestão Viária sabemos também que o tamanho de um traço descontinuo é de 4 metros e com 10 metros de separação entre eles.

Quadro 3 - Dimensões das marcas longitudinais (m)

Linhas	Velocidades		110 - 130 km/h (Auto-Estradas)			90 - 110 km/h (Vias Reservadas)			60 - 90 km/h (Restantes estradas ³)			40 - 60 km/h (Restantes estradas ³ e arruamentos urbanos)		
	RST	NMR	Larg.	Traço	Esp.	Larg.	Traço	Esp.	Larg.	Traço	Esp.	Larg. (a)	Traço	Esp.
Linha contínua	M1	LBC	0,15	-	-	0,12	-	-	0,12	-	-	0,10	-	-
Linha descontinua	M2	LBT	0,15	4,0	10,0	0,12	4,0	10,0	0,12	4,0	10,0	0,10	2,0 ⁺ 3,0 ⁺	5,0 ⁺ 4,0 ⁺

Pressupostos assumidos

Assumimos que a câmara está alinhada com o eixo da estrada. Desta forma, a distorção da perspetiva é basicamente apenas longitudinal, em que por estarmos a ver a estrada de frente, a sua altura é bem menor do que se a estivessemos a ver por cima. Por esta razão, mantemos a largura da imagem transformada igual à largura da região de interesse na imagem original. Se a câmara estivesse rodada mais para a esquerda ou para a direita, teríamos que ajustar a largura da imagem transformada para refletir a distorção lateral. Abaixo tem uma imagem que gerámos com uma ferramenta de IA (Nano Banana) para exemplificar esta distorção na altura que temos:



Segundo a célula anterior, sabemos que o tamanho real da largura de uma via da autoestrada é de 3.75 metros, dos traços longitudinais descontínuos 4 metros e do espaço entre eles 10 metros. Assim, ao contarmos o número de espaços entre os traços na região de interesse, podemos determinar, no plano visto de cima, quantas vezes maior é a altura da região de interesse comparada com a largura das 3 vias. Com isto, conseguimos estabelecer uma relação de escala entre os pixels na imagem e as distâncias reais em metros, mantendo assim uma boa proporção.

Calibração e Parâmetros Reais

De seguida, precisamos de calcular os parâmetros reais do sistema de projeção de modo a conseguirmos fazer a transformação de perspetiva corretamente. Para isso, usamos como **src_points** os 4 pontos que definem a nossa região de interesse no vídeo original.

Manuseamos estes pontos para calcular a largura total em pixels da região de interesse na imagem original (**LARGURA_PERSPETIVA**) pois esta será igual à largura da imagem transformada como já referido anteriormente.

Sabemos que a largura real em metros da região de interesse é igual a 3 vezes (3 vias) a largura de uma via (3.75 metros). A altura real em metros é calculada com base no número de traços e no número de espaços existentes entre eles na região de interesse. Observando visualmente a região de interesse, podemos contar que existem aproximadamente 3 traços e 3.5 espaços entre eles.

Assim, sabendo a altura e largura real em metros, podemos calcular a proporção que poderemos aplicar à imagem transformada de modo a evitar distorções dos veículos na estrada.

Sabendo a **LARGURA_PERSPETIVA** e a **ALTURA_PERSPETIVA**, podemos definir os **dst_points** que serão os pontos que definem o retângulo que representará a vista de topo das imagens transformadas.

Sem este tratamento de dados, caso atribuíssemos valores arbitrários para a largura e altura das imagens transformadas, a escala entre os eixos X e Y não seria a mais consistente, o que levaria a medições não tão precisas de distância e velocidade.

```
In [22]: #Pontos de Origem
src_points = np.float32([p2, p3, p1, p4]) #Superior Esquerdo, Superior Direito,
#print(src_points)

# Definimos a largura e altura da imagem em perspectiva
LARGURA_PERSPETIVA = (p4[0]-p1[0]) #Canto inferior esquerdo - canto inferior dir

n_vias = 3
#largura_real = 3,75m * 3vias = 11,27metros
largura_real = 3.75 * n_vias #metros

tamanho_linha = 4
espaco_entre_linha = 10

#Ao observar a imagem vemos que temos 3 linhas e 3 espaços e meio dentro da roi
altura_real = (tamanho_linha * 3) + (espaco_entre_linha * 3.5)

proporcao_altura_largura = altura_real/largura_real
#print(proporcao_altura_largura)

#round para ficar inteiro
ALTURA_PERSPETIVA = round(LARGURA_PERSPETIVA * proporcao_altura_largura)

# Pontos de Destino (formam um retangulo em vez de um trapézio)
#Estas funções do open cv trabalham com valores em float32 para manterem a precis
dst_points = np.float32([
    [0, 0], #Superior Esq
    [LARGURA_PERSPETIVA, 0], #Superior Dir
    [0, ALTURA_PERSPETIVA], #Inferior Esq
    [LARGURA_PERSPETIVA, ALTURA_PERSPETIVA] #Inferior Dir
])
```

Transformação de Perspetiva (Homografia)

Com os pontos de origem (**src_points**) e destino (**dst_points**) definidos, podemos calcular a **matriz_homografia**, para isso exploramos a função

`cv2.getPerspectiveTransform` da biblioteca OpenCV.

Com a matriz de homografia calculada, aplicamos dentro da função

aplicar_perspetiva_video a transformação H a cada frame do vídeo original, que

percorrido dentro de um ciclo for, aplica esta matriz a todos os frames do vídeo original com o uso de outra função do OpenCV, a `cv2.warpPerspective`.

Esta aplicação da perspectiva é feita segundo a matriz de homografia calculada, sobre o vídeo original com determinado tamanho de saída (**LARGURA_PERSPETIVA** x **ALTURA_PERSPETIVA**)

É de notar que a transformação só é válida para o plano mesmo da estrada em que $Z=0$. Objetos com altura (como os carros) sofrem uma distorção, em que parecem "deitados" ou achatados.

```
In [23]: # Calcula a Matriz de Homografia H com func do cv2
matriz_homografia = cv2.getPerspectiveTransform(src_points, dst_points)

def aplicar_perspetiva_video(lista_frames, matriz, tamanho_saida):
    frames_transformados = []

    for frame in lista_frames:
        #Aplica a transformação H à imagem inteira
        frame_warped = cv2.warpPerspective(frame, matriz, tamanho_saida)
        frames_transformados.append(frame_warped)

    return frames_transformados

# _____ OUTPUT-TESTE _____
video_vista_topo = aplicar_perspetiva_video(video_original_frames, matriz_homogr
reproduzir_frames(video_vista_topo, titulo="Perspetiva Vista de Cima")
```

A reproduzir 'Perspetiva Vista de Cima'
 Pressiona a tecla 'Q' para fechar
 Reprodução terminada.

Configuração das Vias e FPS

Uma vez obtida a vista de topo, a segmentação das vias torna-se muito simples, pois as linhas descontinuas divisórias agora estão verticais e equidistantes. Logo, dividimos a largura das imagens por 3 secções iguais para definir os limites das vias:

- limite_v1 - limite entre a via da esquerda e a do meio
- limite_v2 - limite entre a via do meio e a que está mais à direita.

A função **obter_via** recebe a coordenada x onde o veículo se encontra no frame atual e devolve a via em que este se encontra (1 (esquerda), 2 (meio) ou 3(direita)).

```
In [24]: limite_v1 = int(LARGURA_PERSPETIVA / 3)
limite_v2 = int(limite_v1 * 2)

LIMITES_VIAS = [0, limite_v1, limite_v2, LARGURA_PERSPETIVA]

#Func auxiliar
def obter_via(x):
    #com base no x do veiculo recorna o indice da via em q está
    for i in range(len(LIMITES_VIAS) - 1):
```

```

    if LIMITES_VIAS[i] <= x < LIMITES_VIAS[i+1]:
        return i + 1
    return 0 #retorna 0 se for indefenido

```

Adicionalmente, para o cálculo da velocidade ($v = \Delta d / \Delta t$), é importante sabermos o intervalo de tempo exato entre frames com o numero real de frames por segundo.

Utilizamos `cap.get(cv2.CAP_PROP_FPS)` para extrair o Frame Rate real do ficheiro de vídeo, o que até ao momento ainda não tínhamos realizado. Assim garantimos a precisão temporal. Se fossemos nós a decidir um frame rate, então as velocidades medidas não seriam as reais, já que bastaria aumentar o frame rate (acelerar o video) para as velocidades serem maiores

```

In [25]: #Verificamos qual os fps da imagem
cap = cv2.VideoCapture("AutoEstrada.avi" )

if cap.isOpened():
    fps = cap.get(cv2.CAP_PROP_FPS)
    print(f"FPS: {fps}")
else:
    print("Erro ao abrir o vídeo")

cap.release()

```

FPS: 23.97

Algoritmo de Cálculo de Velocidades

A função **processar_velocidades** integra todo o conhecimento geométrico e físico adquirido. O processo para cada veículo em cada frame é o seguinte:

- Ponto de Contacto: Seleccionamos o ponto $(x, y + h)$ da bounding box original. Este corresponde mais ou menos a um peneu (canto inferior esquerdo) que está realmente em contacto com a estrada, assim garante que o ponto está no plano $Z=0$ para a sua projecção homografica funcionar mais precisamente. O centro da caixa ou o topo não servem tão bem, pois a sua projecção não iria ser tão precisa porque estes pontos não estão em contacto com o chão.
- Transformação: Aplicamos **cv2.perspectiveTransform** apenas a este ponto de contacto, e obtemos as suas coordenadas (x_t, y_t) na vista de topo.
- Identificação da Via: Com base na coordenada x_t , classificamos o veículo na via correspondente.
- Cálculo da Velocidade: Como calcular a velocidade instantânea entre frames consecutivos (t e $t - 1$) introduz ruído devido à pequena "vibração" da boundingbox então implementámos uma média de velocidade para saber a velocidade "instantânea". Calculamos a distância percorrida entre o frame atual e o frame de há 5 instantes atrás (`GAP_FRAMES = 5`). A distância Euclidiana em píxeis (`dist_píxeis`) é convertida para metros (dividir pelos píxeis por metro previamente calculados) e a velocidade é obtida pela fórmula clássica $v = \frac{\Delta d}{\Delta t}$.

```

In [26]: #CALIBRAÇÃO
LARGURA_VIA_PIXEIS = LARGURA_PERSPETIVA/3

#Pixeis por Metro
#Se uma via tem 3,75 metros então se fizermos regra de 3 simples sabemos que 1 m
PPM = LARGURA_VIA_PIXEIS / 3.75

FPS = 23.97 # Frames por segundo do video original

def processar_velocidades(dados_rastreamento, matriz_homografia):
    historico_veiculos = {}
    stats_vias = {1: [], 2: [], 3: []}
    novos_dados_rastreamento = []

    for num_frame, veiculos_frame in enumerate(dados_rastreamento):
        novo_frame_dados = []

        for veiculo in veiculos_frame:
            # veiculo = (x, y, w, h, id_carro)
            x, y, w, h, id_carro = veiculo

            # Ponto de contacto - canto inferior esquerdo que corresponde a uma
            ponto_base = np.array([[x, y + h]], dtype=np.float32)
            #0 perspectiveTransform recebe como parametros array 3d

            # Transformar ponto
            ponto_transf = cv2.perspectiveTransform(ponto_base, matriz_homografi
            #0 opencv está pronto para transformar muitos pontos de uma só vez,
            x_t = ponto_transf[0][0][0]
            y_t = ponto_transf[0][0][1]

            #Histórico
            if id_carro not in historico_veiculos:
                historico_veiculos[id_carro] = []
            historico_veiculos[id_carro].append((num_frame, x_t, y_t))

            # Calcula
            velocidade_kmh = 0.0
            via = obter_via(x_t)

            #Se calcularmos a velocidade entre frames consecutivos, vai haver ma
            #Por isso decidimos, para calular a velocidade instantania, ver a di
            GAP_FRAMES = 5
            hist = historico_veiculos[id_carro]

            if len(hist) > GAP_FRAMES:
                frame_ant, x_ant, y_ant = hist[-1 - GAP_FRAMES] #o ultimo frame(

                #Calc da dist e intervalo tempo
                ponto_atual = (x_t, y_t)
                ponto_antigo = (x_ant, y_ant)

                dist_pixeis = math.dist(ponto_atual, ponto_antigo)#Dist que perc
                dist_metros = dist_pixeis / PPM #Dist que percorreu em metros
                delta_tempo = (num_frame - frame_ant) / FPS #Tempo que passou em

                distancia_km = dist_metros / 1000.0
                tempo_horas = delta_tempo / 3600.0 #3600 seg em 1hora
                velocidade_kmh = distancia_km / tempo_horas

```

```

        stats_vias[via].append(velocidade_kmh)

        nova_label = f"{id_carro}|{int(velocidade_kmh)}km/h"
        novo_frame_dados.append((x, y, w, h, nova_label))

    novos_dados_rastreamento.append(novo_frame_dados)

# OUTPUT
print("VELOCIDADE MÉDIA EM CADA VIA:")
for via, velocidades in stats_vias.items():
    media = sum(velocidades) / len(velocidades)
    print(f"Via {via}: Velocidade Média = {media:.2f} km/h")

return novos_dados_rastreamento

# _____OUTPUT-TESTE_____
dados_com_velocidade = processar_velocidades(dados_com_veiculos_identificados, m
reproduzir_frames(video_original_frames, pontos_roi, dados_com_velocidade, titul

```

VELOCIDADE MÉDIA EM CADA VIA:

Via 1: Velocidade Média = 99.85 km/h

Via 2: Velocidade Média = 78.05 km/h

Via 3: Velocidade Média = 68.47 km/h

A reproduzir 'Resultado Final com Velocidades'

Pressiona a tecla 'Q' para fechar

Reprodução terminada.

Resultado Obtido: Os resultados apresentam-se consistentes com o esperado para um cenário de autoestrada. As velocidades médias situam-se no intervalo típico (50 km/h a 120 km/h) valores limite de circulação dentro de uma autoestrada em Portugal.

Observa-se que a via mais à esquerda (Via 1), tipicamente usada para realizar ultrapassagens, tem uma velocidade média superior às vias mais à direita, o que valida assim a lógica do sistema de contagem de velocidades implementado.

Conclusão

Neste trabalho, desenvolvemos um algoritmo capaz de detetar e contar veículos no vídeo de teste que foi fornecido pelo docente. Utilizamos técnicas de Análise de Movimento e Segmentação de imagens para alcançar este objetivo (recurso às funções da biblioteca OpenCV).

Tendo em conta que tentamos implementar opcionalmente a velocidade de cada veículo e consequentemente a média das velocidades em cada via, sentimos que nesta parte do trabalho houve algumas dificuldades, nomeadamente na calibração da perspetiva da região de interesse. Sentimos também algumas dificuldades na parte da correspondência entre imagens consecutivas, especialmente na gestão de veículos que desapareciam temporariamente da região de interesse bem como as suas reidentificações.

Contudo, acreditamos que o trabalho desenvolvido cumpre os objetivos propostos e demonstra a nossa tentativa de uma boa prática das técnicas lecionadas bem como das funções da biblioteca do OpenCV.

Bibliografia

Jorge, Pedro, Documentação da unidade curricular 2025-2026, Lisboa: ISEL, 2025

<https://docs.opencv.org/4.x/> - Documentação oficial da biblioteca OpenCV

<https://numpy.org/doc/> - Documentação oficial da biblioteca NumPy

<https://www.ccdr-n.pt/storage/app/media/2025/Mobilidade/12SinRodoviaria.pdf> - Manual de Planeamento das Acessibilidades e da Gestão Viária

<https://www.imt-ip.pt/wp-content/uploads/IMTT/Portugues/InfraestruturasRodoviaras/InovacaoNormalizacao/Divulga> - Características Técnicas das Autoestradas em Portugal

<https://www.geeksforgeeks.org/> - Geeks for Geeks

<https://artlist.io/image-to-image-ai/nano-banana> - Nano Banana, Ferramenta de IA para geração de imagens



In []: